



Software Design e System Design

Autoria: Otávio Gonçalves de Santana

Introdução

Sejam muito bem-vindos a esse módulo sobre **Software Design e System Design**. Ao longo dessa jornada, vamos discutir os fundamentos que vão muito além de escrever código funcional. Vamos entender como pensar e construir sistemas de forma consciente, escalável, segura e, acima de tudo, alinhada com os princípios que regem o design de software moderno.

Você, como engenheiro ou engenheira de software, precisa olhar para o sistema como um todo. Precisa entender não só o que o sistema faz, mas **por que** ele é construído de determinada maneira. Esse é o coração do System Design: compreender os componentes, as dependências, os fluxos de dados e as necessidades do negócio. Mais do que codificar, é essencial projetar.

Já em Software Design, vamos nos aprofundar em boas práticas de escrita de código, princípios como coesão, baixo acoplamento, encapsulamento, testabilidade e evolutividade. Falaremos também sobre padrões de projeto (design patterns), boas práticas arquiteturais e sobre como a documentação pode ser uma aliada para manter a consistência do sistema no longo prazo.

Nos primeiros capítulos, discutimos os princípios fundamentais que norteiam o System Design, passando por comunicação entre componentes, documentação eficaz e estruturação de sistemas orientada ao contexto. Depois, mergulhamos no universo do Software Design, onde exploramos práticas de código limpo, aplicabilidade de padrões, visões táticas e estratégicas do DDD (Domain Driven Design) e finalizamos conhecendo ferramentas modernas como o JMolecules, que ajudam a manter a arquitetura viva dentro do código.

Mais do que aprender conceitos, minha proposta aqui é compartilhar experiências, mostrar como esses conceitos se aplicam na prática, no código, nos projetos reais. Quero que você termine esse módulo enxergando o design de sistemas como uma competência essencial, com olhar crítico e sensibilidade técnica para tomar decisões que sustentem a evolução do software ao longo do tempo.

Vamos nessa!

Capítulo 1: Software Design & System Design -

Introdução

Neste capítulo introdutório, quero convidar você a refletir comigo sobre os fundamentos e a importância do Software Design e do System Design na engenharia de software atual. Meu objetivo aqui não é substituir o estudo de arquitetura de software, mas sim complementar esse conhecimento, mostrando onde arquitetura, design e planejamento sistêmico se encontram e se diferenciam.

O Que São Software Design e System Design?

Quando falamos em System Design, estamos tratando do planejamento do sistema como um todo: como os componentes se organizam, como interagem, e quais decisões estruturais guiam o comportamento da aplicação. Já o Software Design está mais próximo do código, das decisões técnicas que tomamos no momento da implementação. Enquanto o primeiro foca na visão macro, o segundo se concentra na execução.

A Importância do "Porquê"

Um ponto que sempre reforço é a transição natural na carreira de qualquer desenvolvedor: deixamos de perguntar "como faço isso?" para começar a questionar "por que estou fazendo isso assim?". É essa mudança que marca a maturidade profissional. E para quem quer chegar aos níveis de Staff Engineer ou Software Architect, entender o "porquê" das escolhas técnicas se torna essencial.

A Complexidade ao Longo das Eras

Divido a evolução do software em cinco grandes eras, cada uma com sua complexidade específica:

1. **Era do Hardware Limitado** – Aqui, o hardware era caro e escasso. Tudo precisava ser feito com o mínimo de recursos.

2. **Era da Concorrência** – Com processadores multinúcleo, enfrentamos desafios para desenvolver softwares realmente concorrentes.
3. **Era dos Sistemas Distribuídos** – Começamos a lidar com múltiplos servidores, APIs, cache e comunicação entre serviços.
4. **Era da Escalabilidade e Distribuição Global** – As aplicações passaram a rodar em vários dispositivos, em vários países, com times distribuídos.
5. **Era da Complexidade Acidental** – Começamos a adicionar ferramentas, camadas e padrões sem avaliar sua real necessidade. Criamos complexidade onde não havia.

Camadas Físicas e Lógicas

Quando analisamos a estrutura dos sistemas, gosto de explicar a diferença entre as camadas físicas (servidores, dispositivos, etc.) e as camadas lógicas (UI, negócio, repositórios, etc.). Essa separação evoluiu com o tempo:

- **Mainframe**: tudo em uma única máquina.
- **Two-Tier**: separação entre cliente e servidor.
- **Three-Tier**: aplicação, cliente e banco de dados separados.
- **Múltiplas Camadas**: estrutura mais modular, conectando diferentes sistemas.

Cada camada que adicionamos busca simplificar a manutenção e o desenvolvimento, mas também adiciona novos desafios.

O Papel da Engenharia de Software

A engenharia de software nasce da necessidade de lidar com a crescente complexidade. E eu gosto de dividir o processo em quatro etapas:

1. **Entender o problema**
2. **Planejar o sistema (System Design)**
3. **Definir a arquitetura (Software Architecture)**
4. **Implementar a solução (Software Design)**

Esse processo é cíclico e se encaixa muito bem com métodos ágeis e sprints.

Definições e Referências

Sempre menciono que há várias definições sobre arquitetura de software. Recomendo autores como Alex Xu, e livros como *Just Enough Software Architecture* e *Building Evolutionary Architectures*. Gosto de começar pelo básico e só depois ir para abordagens mais avançadas como o *Clean Architecture*.

Para mim, arquitetura de software é um equilíbrio entre pessoas e tecnologia. Não se trata apenas de escalabilidade e performance, mas também de cultura, comunicação e estratégia.

A Relação entre System Design, Arquitetura e Design

Costumo organizar as responsabilidades assim:

- **System Design:** visão estratégica e alinhamento com o negócio.
- **Software Architecture:** fronteiras técnicas e decisões estruturais.
- **Software Design:** decisões táticas e implementação.

Mas essas fronteiras são flexíveis. O design afeta a arquitetura, e vice-versa. E tudo deve estar conectado à realidade do projeto e da equipe.

Essa é a base para o módulo. Vamos aprofundar cada um desses temas nos próximos capítulos, sempre com foco em aplicar esses conceitos na prática do desenvolvimento moderno de software.

Capítulo 2: Princípios Básicos de System Design

Neste capítulo, quero compartilhar com vocês os fundamentos que considero essenciais para entender System Design. Vamos partir da estrutura física e lógica dos sistemas até chegar aos componentes que constroem um ecossistema de software moderno. A minha ideia é mostrar como conceitos que muitas vezes parecem abstratos fazem toda a diferença na prática do dia a dia.

Estrutura Física e Lógica: Evolução e Impactos

Com o passar dos anos, a estrutura dos sistemas de software mudou muito. Lá atrás, tínhamos tudo em uma única máquina. Depois passamos a separar cliente, servidor e banco de dados em camadas físicas distintas. E isso trouxe novas possibilidades – conseguimos escalar partes do sistema de forma independente – mas também surgiram novos desafios, como orquestração, sincronização de dados e controle de falhas.

Na lógica, também houve mudança: saímos de sistemas monolíticos para estruturas mais modulares. E isso melhorou muito a manutenção e a testabilidade, mas quando exageramos nas camadas, criamos complexidade desnecessária.

Componentes: Unidades Essenciais do Design

Sempre explico que um componente pode ser qualquer unidade lógica ou física do sistema – interface, gateway, mensageria, repositório. O importante é que ele tenha responsabilidade clara e bem definida. No System Design, esses componentes são nossos blocos de construção. Precisamos pensar neles de forma estratégica, avaliando reusabilidade, deploy independente e isolamento.

Front-End e Back-End: Papéis e Distribuição

Quando falo sobre front-end e back-end, gosto de mostrar que há várias formas de distribuir responsabilidades. O front pode estar no cliente, no servidor ou nos dois. Usar React, Angular ou JSF muda bastante o comportamento da aplicação e também a performance.

Já o back-end é o núcleo das regras de negócio. Quando separamos bem as camadas, podemos evoluir cada uma de forma mais independente, o que facilita a manutenção e o crescimento do sistema.

Escalabilidade: Vertical, Horizontal e Seus Trade-offs

Escalar significa lidar com mais usuários, mais dados ou mais requisições. Existem duas formas principais:

- **Vertical:** melhorar a máquina.
- **Horizontal:** adicionar novas máquinas.

Escalar verticalmente é mais simples, mas tem um limite. Horizontalmente, conseguimos crescer mais, mas o sistema precisa saber lidar com isso. Sempre analiso os trade-offs entre custo, complexidade e disponibilidade antes de decidir.

Latência e Throughput: Métricas de Desempenho

Duas métricas que sempre levo em consideração são:

- **Latência:** tempo entre requisição e resposta.
- **Throughput:** quantas requisições consigo processar por segundo.

Essas métricas me ajudam a decidir quando mudar uma arquitetura, reescrever uma parte do sistema ou adotar uma tecnologia diferente.

Considerações sobre Abstrações e Reuso

Sou defensor da simplicidade. Abstrações e camadas só fazem sentido quando resolvem um problema real. Muitas vezes vejo projetos cheios de camadas que só dificultam o entendimento. O mesmo vale para o reuso: reutilizar código é ótimo, mas não a qualquer custo. Cada camada precisa existir com um propósito claro.

Alinhamento de Times e Flexibilidade

System Design também tem tudo a ver com como os times estão organizados. Em empresas com muitos times especializados, uma separação clara entre camadas ajuda bastante. Já em times pequenos, precisamos ser mais flexíveis. Sempre reforço que não existe uma arquitetura perfeita – existe a arquitetura que melhor se adapta àquele contexto.

Conclusão

System Design é sobre enxergar o sistema como um todo. É sobre como as partes se encaixam, como crescem juntas e como lidam com falhas e mudanças. Esse capítulo serve como base para as próximas discussões, onde vamos mergulhar nos componentes, formas de comunicação, persistência e os desafios reais de construir sistemas escaláveis e distribuídos.

Capítulo 3: System Design e Documentação:

Neste capítulo, quero mostrar para vocês como a documentação é uma parte essencial do System Design. Costumo dizer que projetar um sistema não se trata apenas de escrever código ou desenhar arquitetura – também é saber comunicar tudo isso de forma clara. Documentar é garantir que o conhecimento não se perca, que decisões sejam justificadas e que o time possa crescer com mais fluidez.

Por que Documentar System Design?

Sempre que me perguntam se vale a pena documentar, a minha resposta é: absolutamente sim. Com uma documentação bem feita, economizamos tempo de reuniões, melhoramos o alinhamento entre os times e aceleramos o onboarding. Documentar também é uma forma de tornar o sistema mais escalável do ponto de vista humano – algo que muitas vezes é negligenciado.

Formatos e Modelos de Documentação

UML (Unified Modeling Language)

A UML ainda é bastante útil, especialmente em alguns diagramas que uso com frequência:

- **Diagrama de componentes:** ideal para visualizar a arquitetura modular.
- **Diagrama de sequência:** excelente para entender fluxos de requisições.
- **Diagrama de atividades:** ajuda a modelar interações e workflows.
- **Diagrama de caso de uso:** útil para mapear a relação entre atores e funcionalidades.

Diagrama de classe, confesso, uso muito pouco. É mais comum em contextos com processos rígidos, como outsourcing.

C4 Model

Gosto muito do C4 Model, criado pelo Simon Brown. Ele traz uma abordagem de zoom progressivo, como no Google Maps:

- **Contexto:** visão macro do sistema e seus usuários.

- **Containers:** visão dos serviços, APIs, bancos e front-ends.
- **Componentes:** detalhamento interno dos containers.

Na prática, geralmente uso os dois primeiros níveis para System Design. Os componentes reservo para arquitetura técnica mais profunda.

O interessante é que o C4 foi pensado para ser versionado como código. Ferramentas como DrawIO fogem um pouco dessa ideia. Prefiro linguagens como Markdown ou AsciiDoc para manter os diagramas junto do repositório, de forma rastreável.

Architecture Decision Record (ADR)

Os ADRs são outra ferramenta que uso bastante. Eles registram decisões arquiteturais com contexto, justificativa e consequências. Um exemplo: decidimos adotar Java 21. Isso vira um ADR. No futuro, ao migrar para Java 25, criamos outro ADR e deprecia-se o anterior. É um histórico vivo do pensamento arquitetural da equipe.

Documentação como Código (DocOps)

Sou fã da prática de manter a documentação junto do código. Essa abordagem traz vários benefícios:

- Versionamento no Git
- Revisão por pull requests
- Padronização do conhecimento

As ferramentas que mais utilizo são:

Markdown

Simples, direto e ideal para começar. Funciona bem com GitHub Pages e IDEs.

AsciiDoc

Mais poderoso, ideal para documentações maiores. Dá para gerar PDFs, HTMLs e organizar bem o conteúdo com Table of Contents. Uso bastante quando quero uma documentação mais estruturada e durável.

Ferramentas de Diagramação

PlantUML

Tradicional e confiável. Útil para diagramas de sequência e componentes.

Mermaid

Minha favorita. Moderna, suporta vários tipos de diagrama e funciona muito bem com documentação versionada.

Structurizr

Ferramenta feita pelo próprio criador do C4 Model. Ideal para quem quer aplicar a filosofia do C4 ao pé da letra.

Boas Práticas e Exercícios

Costumo propor exercícios como:

- Documentar o fluxo de uma API REST
- Criar um diagrama de estados para um cartão de crédito
- Modelar um sistema simples com o C4

A dica de ouro é: versionem tudo. Diagrama, Markdown, ADR, tudo. A rastreabilidade é uma aliada poderosa.

Conclusão

Documentar o System Design é uma forma de manter o projeto sustentável e acessível. Ajuda no alinhamento técnico e organizacional, acelera o crescimento do time e garante que boas decisões não se percam com o tempo. Espero que

esse capítulo tenha te mostrado o quanto a documentação é estratégica, e não apenas uma burocracia.

Capítulo 4: Introdução a Banco de Dados na Perspectiva de System Design

Neste capítulo, quero compartilhar com vocês a minha visão sobre bancos de dados sob a ótica de System Design. Vamos partir de conceitos históricos e filosóficos até chegarmos às práticas modernas de arquitetura orientada a dados. Ao longo da aula, discutimos os principais tipos de bancos de dados, seus paradigmas, características de escalabilidade e aplicações práticas no dia a dia de um arquiteto de software.

A Importância do Banco de Dados

Sempre começo essa discussão com uma reflexão histórica. O ser humano sempre buscou formas de armazenar e preservar informações. Essa é a essência do banco de dados: garantir que o conhecimento seja guardado com segurança e esteja disponível no momento certo. Desde inscrições rupestres até sistemas distribuídos modernos, essa necessidade moldou a nossa forma de criar tecnologia.

Em aplicações modernas, o banco de dados representa o estado da aplicação. Mesmo quando trabalhamos com aplicações stateless, é nos bancos de dados que os dados realmente vivem. E esses dados são essenciais não só para o funcionamento da aplicação, mas também para geração de insights de negócio. Um caso interessante que sempre menciono é o de um banco brasileiro que descobriu um padrão de consumo de vinhos entre seus clientes, o que originou uma nova frente de negócios.

Classificação dos Bancos de Dados

Gosto de dividir os bancos de dados em dois grandes grupos: **relacionais** e **não-relacionais**. Cada um tem suas características, vantagens e desafios.

Relacional

Os bancos relacionais são baseados em normalização e estão presentes desde os anos 1970. São robustos, confiáveis e maduros. Costumam escalar verticalmente – ou seja, aumentando o poder de uma única máquina. Uma grande vantagem é a padronização: o conhecimento de SQL está amplamente difundido e isso reduz a carga cognitiva das equipes.

Não-Relacional (NoSQL)

Nos bancos não-relacionais, encontramos maior flexibilidade e escalabilidade horizontal. Gosto de classificá-los assim:

- **Chave-Valor**: estrutura simples, como um mapa. O Redis é um exemplo. É ótimo para sessões e cache.
- **Wide Column**: tem uma chave com várias colunas. Cassandra é um dos mais conhecidos e é usado, por exemplo, pela Netflix.
- **Documentos**: estrutura baseada em JSON ou XML. MongoDB é um exemplo clássico. Ideal para aplicações móveis e catálogos.
- **Grafos**: representam relacionamentos complexos. O Neo4j é um exemplo que uso bastante para ilustrar motores de recomendação ou análise de fraude.

Também menciono outros modelos que estão ganhando espaço, como **Time Series**, **Vector Database** e os bancos **Multi-Model**, como o ArangoDB, que combinam vários paradigmas.

Persistência Poliglota

Aqui trago uma analogia com os políglotas. Em vez de escolher um banco de dados e forçar todos os cenários a usá-lo, prefiro adotar a persistência poliglota: usar o banco certo para cada situação. Por exemplo:

- **Grafo** para recomendação
- **Chave-Valor** para cache
- **Wide Column** para logs
- **Documento** para leitura otimizada
- **Relacional** como base confiável

Evitem soluções únicas para todos os problemas. O ganho vem da combinação certa.

Estratégias de Armazenamento e Arquitetura de Dados

Sharding

É uma técnica que uso quando preciso distribuir dados horizontalmente. Traz ganhos em performance e escalabilidade, mas também exige atenção com backups e consistência.

Redundância e Replicação

Existem dois modelos principais:

- **Master-Slave:** apenas um nó escreve, os outros replicam. Alta consistência.
- **Masterless:** todos os nós escrevem. Alta disponibilidade, com risco de perda de consistência.

Teorema do CAP

Sempre falo do CAP porque ele mostra que não dá para ter tudo ao mesmo tempo: consistência, disponibilidade e tolerância a falhas. A escolha depende da prioridade da aplicação. Por exemplo:

- **Relacional:** consistência + disponibilidade
- **MongoDB/Redis:** consistência + tolerância a falhas
- **Cassandra:** disponibilidade + tolerância a falhas

Cache

Cache é essencial para performance, mas deve ser usado com cuidado. Nunca uso cache para dados voláteis como saldo bancário. Mas nome de usuário, por exemplo, é um bom candidato.

ETL, ELT, Data Warehouse e Data Lake

ETL

Extraímos, transformamos e só então carregamos os dados. É o processo mais comum em relatórios empresariais.

ELT

Extraímos, carregamos e depois transformamos. Ideal quando os dados precisam ser analisados de formas diferentes por cientistas de dados.

Data Warehouse

Estrutura para armazenar dados estruturados, voltada para análise com performance.

Data Lake

Repositório de dados brutos e não estruturados. É mais barato, mas exige cuidados na curadoria.

Mesmo não sendo o foco direto do curso, esses temas influenciam diretamente o System Design.

Mapeamento Objeto-Relacional (ORM)

Esse é um ponto importante. Aplicações orientadas a objeto e bancos relacionais têm paradigmas diferentes. O ORM ajuda, mas é preciso entender que há impactos na performance e nas decisões de modelagem.

Considerações Finais

Sempre reforço que a escolha do banco de dados deve ser feita com base nos objetivos da aplicação. Considerem escalabilidade, consistência, performance e flexibilidade. E, acima de tudo, evitem dogmas. Combinar tecnologias de forma inteligente é o que gera soluções resilientes e eficazes.

Espero que este capítulo tenha ajudado a ampliar a visão sobre como bancos de dados se integram ao System Design. Nos próximos capítulos, vamos aprofundar ainda mais essa relação com outros elementos da arquitetura.

Capítulo 5: Comunicação entre Componentes (System Design)

Olá a todos! Hoje quero conversar com vocês sobre um tema que é crucial em System Design: a comunicação entre componentes. Esse tópico, embora não substitua a parte de arquitetura, complementa com profundidade a maneira como projetamos sistemas robustos, resilientes e escaláveis.

O Papel da Comunicação no Design de Sistemas

Costumo dizer que sistemas modernos não vivem isolados. Eles precisam se comunicar com terceiros, outros setores internos e serviços externos. Já trabalhei em e-commerces onde existiam áreas de estoque, visualização, pagamento e machine learning – todos trocando informações o tempo inteiro.

Essa comunicação pode acontecer de forma síncrona ou assíncrona, e é sobre isso que vamos tratar aqui. O foco é mostrar como garantir robustez, disponibilidade e resiliência nas interfaces entre componentes.

Comunicação Síncrona

REST e gRPC

REST continua sendo a forma mais comum de comunicação entre aplicações, especialmente entre empresas (B2B). A documentação, a popularidade e as ferramentas fazem dele um padrão de fato. Uso muito Swagger (OpenAPI) para documentar APIs – é como um WSDL para os desenvolvedores mais jovens.

Os verbos HTTP são parte importante dessa estrutura: GET, POST, PUT, DELETE. Por exemplo, o PUT pode ser usado tanto para update quanto para inserção (upsert). O POST normalmente insere, e o DELETE remove.

O gRPC, por outro lado, tem crescido muito. É uma solução do Google que aproveita HTTP/2 e permite comunicação full duplex. Uso bastante em ambientes internos, porque exige controle mais fino de conexões e portas, mas oferece baixa latência e alto desempenho.

Exemplos Práticos

Imagine um banco pequeno querendo emitir cartões para seus clientes. Ele contrata um serviço externo para isso. A comunicação se dá por REST: o backend envia uma ordem de emissão, recebe uma confirmação e armazena no banco de dados. Mais tarde, recebe um webhook com status atualizado do cartão.

Internamente, posso usar gRPC entre serviços, por exemplo, quando um serviço de estoque precisa confirmar se um produto está disponível. Isso permite separar bem as responsabilidades e manter a eficiência da comunicação.

Comunicação Assíncrona

A comunicação assíncrona é poderosa, principalmente para escalabilidade e resiliência. Uso RabbitMQ, Kafka e padrões como Observers com CDI para desacoplar componentes. Já atuei em sistemas onde era necessário processar recargas de cartões pré-pagos com alta demanda e longos tempos de resposta. Nesse cenário, o uso de filas e retries é fundamental.

Estratégias de Resiliência

A resiliência em sistemas distribuídos exige mecanismos como:

- Retries com backoff exponencial
- Dead letter queues
- Estratégias de failover

Lidar com falhas é parte do jogo. Já precisei garantir que, se um pagamento falhar, ele será reprocessado automaticamente até funcionar ou ser enviado para análise manual.

Event-Driven Design

Trabalhar com eventos me permite criar sistemas desacoplados. Quando recebo uma notificação da DHL sobre a entrega de um pedido, lanço um evento. O serviço de comunicação envia a notificação ao usuário, o serviço de pedidos atualiza o status e o dashboard do back-office reflete tudo isso. Cada componente entende o evento conforme sua necessidade.

Já apliquei isso em fintechs que lidam com criptomoedas e moedas fiduciárias. Cada evento, como um pagamento recebido, é processado por serviços distintos para gerar bônus, atualizar saldos ou ajustar o dashboard.

Contratos Fortes de API

Sempre reforço a importância de contratos claros e consistentes entre sistemas:

- Use Swagger para REST e Proto para gRPC
- Padronize formatos (ex: representação de dinheiro)
- Estabeleça validações e versionamento de API

Já vi várias empresas com múltiplas representações de dinheiro dentro da mesma aplicação. Por isso, mantenha um padrão e facilite a vida do time.

Boas Práticas de Comunicação

Aqui vão algumas boas práticas que aplico no dia a dia:

- **Versionamento:** URL com /v1, /v2 e pacotes separados.
- **Idempotência:** especialmente em operações sensíveis como pagamentos.
- **Segurança:** autenticação, autorização e validação rigorosa.
- **Observabilidade:** logs, métricas e tracing (Prometheus, Grafana, ELK, OpenTelemetry).
- **Resiliência:** ferramentas como Resilience4j, padrões como Circuit Breaker.

Replicação e Disponibilidade

Serviços como Amazon Multi-AZ ajudam com replicação de dados. Em bancos, isso evita perda de informação e melhora a performance. Uso estratégias ativas e passivas de failover, dependendo do cenário. O Kubernetes facilita muito esse gerenciamento com health checks (readiness e liveness probes).

Conclusão

A comunicação entre componentes é a espinha dorsal de sistemas modernos. Saber quando usar REST, gRPC ou mensageria faz toda a diferença. Mais importante ainda é garantir contratos claros, boas práticas de resiliência e observabilidade.

Compartilhei aqui experiências que vivi em fintechs e e-commerces, para mostrar como esses conceitos funcionam na prática. E na próxima aula, vamos trazer estudos de caso reais para aplicar tudo isso com mais profundidade.

Nos vemos lá!

Capítulo 6: Exemplos e Visão de Geral de Cases de System Design

Opa, galera! Tudo bem? Hoje vamos para uma parte que gosto bastante: exemplos práticos de System Design. Quero compartilhar com vocês alguns cenários que vivi e aproveito para incentivar vocês a praticarem. Nada substitui a prática. Peguem aquele livro *System Design Interview*, leiam, pratiquem,

calculem armazenamento, estimem usuários e tráfego. Esse capítulo serve para te inspirar e mostrar como é possível conectar teoria com prática.

Caso 1 – E-commerce como Bridge Multicanal

Trabalhei em um projeto onde criamos um broadcast multichannel product. A ideia era funcionar como uma ponte entre e-commerces. O usuário entrava, procurava, por exemplo, um iPhone 16, e o sistema listava as ofertas mais baratas entre diversos e-commerces. Mas não era só isso – eu também vendia do outro lado. Informava para os e-commerces que o concorrente X estava com um preço melhor. Era uma venda dupla.

Comecei esse projeto como sempre gosto: entendendo o problema, definindo o contexto no C1 do C4 Model. Não se esqueçam, o modelo C4 é poderoso – C1 mostra o contexto, C2 os containers, C3 os componentes e C4 o código. Neste caso, desenhei o C1 com três usuários principais: o vendedor (via API, FTP ou e-mail), o operador interno (que fazia o monitoramento e configurações) e o sistema que puxava atualizações e notificava os e-commerces e usuários.

O C2 foi ainda mais interessante. Variações no input eram comuns. Alguns vendedores usavam REST, outros mandavam e-mails, arquivos CSV, etc. Eu tinha que estar preparado para tudo. Criei uma camada de ingestão de dados (ETL), que mandava para uma fila – no caso, RabbitMQ. Um processador consumia isso, normalizava os dados e armazenava em dois lugares: um raw data (que depois virou um Data Lake) e um banco de dados normalizado. Usamos o Druze como motor de normalização, que funcionava muito bem.

As informações mais quentes iam para o cache (Redis). Com isso, era possível mostrar os produtos mais baratos ao usuário e também enviar notificações para os concorrentes.

Lições aqui? Diversidade de integração, necessidade de escalabilidade e importância de normalização e cache. E claro, observabilidade! Tinha meu Error Handler e monitoramento para garantir que tudo estava funcionando. Se a ingestão de dados caísse, significava que os preços não estavam atualizados. E isso era inaceitável.

Caso 2 – Sistema de Reservas

Outro projeto envolveu um sistema de reservas, como um Uber. Segui a mesma lógica: C1, C2 e fluxo. Comecei entendendo o negócio: quem é o usuário, o que eu vou desenvolver e de quem dependo. Pagamento, mapa e notificação eram dependências externas. Essas precisavam de contratos bem definidos desde o início.

Gosto do C4 Model porque ele me ajuda a trazer o negócio para o centro da discussão. No C2, detalhei os serviços internos, os contratos com terceiros e as dependências. O C3, normalmente, não faço porque é muito focado na arquitetura interna. Prefiro investir meu tempo nos diagramas de fluxo e na visão de componentes, que são mais úteis nesse estágio.

Caso 3 – Rede Social Corporativa

Também trabalhei em uma rede social corporativa focada em quebrar silos de conhecimento. Fizemos tudo baseado na Google Cloud. Usamos o Elasticsearch (versão gerenciada do Google) e aproveitamos o Data Lake da própria plataforma. O sistema era simples: tínhamos processamento, notificação, raw data e machine learning.

O interessante aqui era o modelo de relevância: como priorizar a informação para o usuário. Uma comunicação do CTO, por exemplo, era mais importante que uma notificação do time de pagamentos. Com isso, usamos machine learning para ordenar o que era relevante.

Considerações Finais e Recomendações

Todos esses casos têm algo em comum: foco no negócio, clareza de contexto, atenção aos pontos críticos e comunicação clara com as partes envolvidas. Outra dica importante: diferencie o que é seu (core), o que é genérico e o que é suporte. O core merece mais atenção e investimento.

Usei Amazon para notificações e armazenamento de mídia (S3), e serviços externos para autenticação e analytics. Isso me permitia concentrar o esforço no produto principal. Por isso, defina seus limites desde o início.

Para finalizar, o que recomendo para todos que querem dominar System Design:

- Estude o *System Design Interview – Volume 1*
- Modele cenários com C4
- Discuta com outros profissionais, critique, melhore
- Priorize o core do seu sistema
- Pratique, pratique, pratique

Essa foi a primeira parte do módulo de System Design. Abordamos componentes, comunicação, camadas, banco de dados e agora, os exemplos. A seguir, vamos mergulhar em Software Design com uma nova perspectiva.

Valeu por acompanhar até aqui. Até a próxima!

Capítulo 7: Visão Geral de Práticas de Código

Fala, pessoal! Que bom estar com vocês novamente. Depois de mergulharmos no System Design, agora entramos em uma nova fase do nosso módulo: o Software Design. E eu quero começar essa parte com algo essencial, que é a forma como escrevemos código. Nesse capítulo, vamos falar sobre as boas práticas, princípios, erros comuns e o impacto real de um bom ou mau código no dia a dia de um desenvolvedor.

Na minha experiência, escrever um bom código é mais do que uma questão de estética ou opinião. É questão de produtividade, manutenibilidade e sustentabilidade do software. Para começar, pensem nisso: 42% do tempo semanal de um desenvolvedor é gasto lidando com código ruim. E código ruim não é só aquele que está desorganizado. É aquele que é difícil de entender, de testar, de modificar.

Eu gosto de comparar com aquela teoria das janelas quebradas. Se você entra em um ambiente bagunçado, é mais fácil deixar mais bagunçado ainda. No código, acontece o mesmo. Um sistema com baixa qualidade encoraja ainda mais a escrita de código ruim.

Código ruim também custa caro. Em termos de tempo, retrabalho e produtividade. Um código bem escrito tem retorno de investimento. Mesmo que isso nem sempre seja entendido pela gestão. Um exemplo claro disso foi quando pedi um mês para refatorar um sistema em uma fintech. Foi difícil conseguir essa autorização, mas o ganho foi enorme: redução de bugs, tempo de resposta e aumento na velocidade de entrega.

Boas práticas de escrita de código incluem princípios como:

- **KISS (Keep It Simple, Stupid):** simplicidade é sofisticação. Um código mais simples é mais fácil de entender e manter.
- **YAGNI (You Aren't Gonna Need It):** pare de tentar prever o futuro. Escreva o código que você precisa agora, não o que imagina que pode precisar.
- **DRY (Don't Repeat Yourself):** evite repetições. Código duplicado é um convite para inconsistências e bugs.

Outra prática que sempre recomendo é o uso de tipos mais ricos em vez de tipos primitivos quando for necessário. Por exemplo, em vez de representar um CPF como uma string, crie um tipo CPF que encapsula validações e regras de negócio.

Organização também é fundamental. Eu prefiro organizar meu código por funcionalidades (Package by Feature) em vez de camadas (Package by Layer). Isso facilita muito a evolução e a migração para microserviços.

Evite heranças desnecessárias. Prefira composição sempre que possível. Herança mal aplicada quebra encapsulamento e traz dependências desnecessárias.

Outro ponto importante é a Lei de Demeter. Evite acessos encadeados como `order.getCustomer().getAddress().getZipCode()`. Isso aumenta o acoplamento e torna o código frágil.

Falando em acoplamento e coesão, nem todo acoplamento é ruim. Em alguns casos, acoplar é necessário e faz sentido. O importante é ter alta coesão e baixo acoplamento, na medida do possível.

Não posso deixar de falar sobre as classes "Deus". Todo mundo já criou um `UserService` que cresceu demais. Quando isso acontece, é hora de quebrar em casos de uso, aplicar encapsulamento de verdade e melhorar a separação de responsabilidades.

Encapsulamento, aliás, não é só colocar getters e setters. É garantir que o estado do objeto só possa ser modificado por ele mesmo de forma segura. E eu vejo muito problema causado por anotações como `@Data` do Lombok, que expõem demais as entidades.

Sobre os princípios SOLID:

- **S**: responsabilidade única.
- **O**: aberto para extensão, fechado para modificação.
- **L**: substituição de Liskov é um dos mais quebrados.
- **I**: segregação de interfaces.
- **D**: inversão de dependência.

Código de qualidade traz agilidade. Projetos com código limpo entregam mais rápido, com menos bugs e com onboarding mais fácil. Tudo isso está em diversos relatórios como da Microsoft, McKinsey, e Google.

E sim, TDD ajuda. Não acho que seja obrigatório sempre, mas é ótimo para novos cenários ou quando você quer garantir qualidade desde o início.

Por fim, lembre-se: refatorar é parte do trabalho. Software degrada com o tempo, mesmo que você não queira. Refatorar ajuda a manter a evolução e combater a erosão do software.

Nos vemos no próximo capítulo!

Capítulo 8: Visão Geral das Práticas de Código na Prática

Fala, pessoal! Dando continuidade, agora a gente vai aplicar na prática tudo aquilo que vimos sobre boas práticas de código. Estou aqui com Java SE, utilizando Java 21 no projeto. Uso algumas dependências, e vou trabalhar com Jakarta CDI, embora tudo o que vou mostrar aqui também sirva para quem trabalha com Spring.

Como comentei antes, eu sou um dos responsáveis pela especificação do CDI desde a versão 2.0, então tenho um carinho especial por ela. Mas os conceitos valem para qualquer framework. Então vamos para um exemplo prático.

Comecei criando um `PaymentService`, com uma implementação chamada `PaymentServiceImpl`, e usei CDI para injetar esse serviço. Até aqui, nada fora do comum. Mas o ponto é: nem sempre você precisa começar com uma interface. Na verdade, minha recomendação é: comece com uma classe simples. Só crie uma interface se houver necessidade real. Se o seu IDE já consegue extrair isso para você automaticamente, melhor ainda. Isso é o que eu chamo de "processo reverso": começar simples e evoluir conforme a necessidade.

Outro ponto crítico é encapsulamento. Às vezes, a gente se engana achando que só porque tem getters e setters privados, o código está bem encapsulado. Um exemplo clássico é o uso de coleções: se você tem um `List<Member>` em uma classe `Team` e disponibiliza um getter que retorna essa lista diretamente, você está expondo o estado interno do objeto. Isso é uma falha de encapsulamento.

A solução é trabalhar com métodos que operam sobre esse estado. Por exemplo, criar um método `addMember()` dentro do `Team` e retornar uma lista imutável com `Collections.unmodifiableList()`. Isso sim é encapsular corretamente.

Um erro comum também é colocar setters desnecessários, como no caso de um `id` que é autogerado pelo JPA. Se não faz sentido alterar esse valor, não exponha um setter. Crie modelos ricos, à prova de falhas.

Outro ponto relevante é a separação entre entidades do negócio e entidades do banco. Alguns desenvolvedores preferem criar uma entidade para o banco (como `PlayerDatabase`) e outra para o modelo de domínio (`Player`). Eu, particularmente, acho isso muito verboso. Prefiro criar objetos de transferência (DTOs) quando preciso expor dados em APIs, como `UserDTO` ou `UserResponse`, controlando melhor o que é exposto.

Falando em organização, vamos comparar dois estilos: `package-by-layer` e `package-by-feature`. O primeiro é aquele clássico: `controller`, `service`, `repository`...

Tudo separado por tipo. O segundo agrupa tudo por funcionalidade: tudo relacionado a User fica junto, tudo sobre Payment também. Gosto muito do package-by-feature, porque facilita a evolução para microserviços e mantém os recursos do mesmo domínio juntos, o que é bem mais organizado.

Outro exemplo que queria trazer é sobre tipos. Em vez de usar BigDecimal e String para representar valores monetários e moedas, recomendo fortemente criar tipos como Money, Email, CPF, etc. Isso aumenta a coesão, melhora a validação e evita erros. Quando usamos tipos primitivos, é muito fácil esquecer alguma validação. Tipos ricos encapsulam essas regras de forma centralizada.

Falhas de consistência também são falhas de segurança. Um email com valor "banana" não é seguro, mesmo que não gere uma exceção. Tipos ajudam a garantir que dados errados não entrem no sistema.

E para fechar, um recurso subutilizado é o enum. Ele é excelente para representar valores fixos e comportamentos. Por exemplo, você pode ter um enum Operator com métodos abstratos execute(double a, double b) implementados por SUM, SUBTRACT, etc. Isso permite usar polimorfismo dentro do enum e deixar o código muito mais limpo.

A ideia aqui é mostrar como todas aquelas boas práticas ganham vida no código real. Espero que esses exemplos tenham ajudado a visualizar melhor os princípios que discutimos. Vamos continuar evoluindo juntos. Até a próxima!

Capítulo 9: Design Patterns

Olá, pessoal! Bom dia, boa tarde, boa noite! Aqui é o Otávio Santana, e agora vamos entrar em um tema muito importante dentro de Software Design: os famosos Design Patterns.

Nas aulas anteriores, a gente falou bastante sobre motivação, sobre os princípios de escrita de código, TDD, refatoração, cobertura de testes, e por aí vai. Agora chegou a hora de dar nomes aos bois. Vamos falar sobre aqueles "templates

reutilizáveis" de soluções que muitos chamam de padrões de projeto. No fundo, eles são comportamentos observáveis e recorrentes que podem ser nomeados e aplicados em diferentes contextos.

Uma analogia que gosto de usar é a musical. Aprendemos escalas, harmonia, ritmo. Design Pattern é como montar música com esses elementos. Quando você aplica bem os princípios, naturalmente vai se deparar com soluções que têm nome: Singleton, Strategy, Observer, etc. E às vezes, sem perceber, você já usou um desses patterns.

Recomendo fortemente a leitura do livro *Head First Design Patterns*. Ele explica de forma simples e eficaz. Fala sobre abstração, sobre encapsular o que varia, e sobre como esses princípios nos levam à construção natural dos padrões. Outro clássico é o *Design Patterns: Elements of Reusable Object-Oriented Software*, do quarteto conhecido como GoF (Gang of Four). São 23 padrões divididos em categorias: criação, estrutura e comportamento.

Vamos então revisar os principais:

Singleton

É o Highlander: só pode existir um. Apesar da discussão sobre ser um antipattern, ele é muito útil quando bem aplicado. Por exemplo, em pool de conexões, configurações globais, objetos caros que precisam ser reutilizados. Em ambientes com CDI, se você define um bean como `@ApplicationScoped`, você já tem um Singleton.

Uma vez, num projeto com alta carga de processamento, havia uma SPI sendo recriada milhares de vezes. Só de cachearmos essa SPI como Singleton, economizamos 600MB de memória e ganhamos uma ordem de grandeza de performance absurda.

Factory

Permite criar objetos sem expor a lógica de criação. Muito útil para objetos complexos ou com regras de validação. Você pode ter fábricas especializadas

para diferentes cenários: pizza italiana, vegetariana, etc. Ele promove o Open-Closed Principle e facilita o polimorfismo.

Strategy

É um dos padrões que mais usamos. Sabe aquele monte de if-else? Strategy resolve isso de forma elegante. Exemplo clássico: pagamentos com PayPal, Stripe, Pix... Cada um pode ser uma implementação diferente. É só trocar a strategy, sem modificar o código existente.

Observer

Permite desacoplar quem emite um evento de quem o consome. Com CDI, é muito simples utilizar esse padrão. É ideal para notificar múltiplos componentes quando algo acontece. Exemplo: uma alteração em um usuário que deve gerar logs, notificações, e reprocessamentos em outros módulos.

Builder

Útil para criar objetos complexos com muitas validações. Evita construtores enormes e torna o código mais fluente. O Builder facilita a aplicação de regras de negócio diretamente na construção do objeto. DSLs e Fluent APIs são uma evolução natural do Builder.

Prototype

Permite clonar objetos complexos. Pouco usado, mas útil quando você precisa fazer cópias profundas sem reconstruir tudo do zero. Em Java, é comum usar Cloneable, embora com moderação.

Adapter

Excelente para integração com sistemas legados. Ele permite adaptar interfaces antigas para formatos modernos. Muito usado ao integrar APIs ou modernizar sistemas antigos.

Decorator

Permite adicionar funcionalidades sem alterar o código original. Exemplo: um gerente é um trabalhador com responsabilidades extras. Com CDI, você consegue compor comportamentos sem herança, apenas com composição.

Proxy

Útil para caching, lazy loading, controle de acesso, logging. CDI e Spring utilizam proxies o tempo todo. No CDI, isso é feito de forma mais rica, na minha opinião. O Spring usa muito AspectJ, que também é poderoso, mas o CDI, com sua evolução para soluções reflectionless, está anos à frente.

Considerações Finais

Lembrem-se: padrões não são receitas prontas. São soluções elegantes para problemas recorrentes. Se você aplicar bem os princípios, naturalmente chegará a esses patterns. Leiam os livros, pratiquem. Um bom engenheiro é aquele que tem bom repertório técnico e cultural. Assim como a gente confia num médico estudioso, o mercado confia em engenheiros que estudam, praticam e têm bagagem.

Na próxima aula, vamos para a prática desses padrões. Nos vemos lá!

Capítulo 10 - Design Patterns - Visão Prática:

Fala pessoal, tudo bem? Vamos continuar nossa jornada com uma abordagem mais prática dos Design Patterns, focando em como aplicá-los com CDI, Spring ou até mesmo usando Java puro. A ideia aqui é entender como usar os padrões de projeto no dia a dia, evitando acoplamentos desnecessários, melhorando a extensibilidade e tornando nosso código mais limpo.

Começo falando sobre o Singleton. No CDI, você consegue aplicar esse pattern com muita facilidade. Basta definir seu bean com `@ApplicationScoped` e ele já se comporta como Singleton. Um ponto importante: o CDI trabalha com injeção *lazy*, ou seja, só injeta quando for realmente necessário. Mas, se você quiser, também é possível configurar como *eager*. Eu já trabalhei em projetos onde, ao

usar Singleton corretamente, reduzi 600MB de consumo de memória só por evitar recriações desnecessárias de componentes.

Um pattern que uso com muita frequência é o Strategy. Ele resolve bem aquele problema clássico de código cheio de if-else. Por exemplo, em um `PaymentService`, é comum ter lógicas diferentes para cada meio de pagamento: PayPal, Cartão de Crédito, Pix... Ao invés de colocar todas essas lógicas em uma classe só, extraio uma interface `PaymentService`, crio implementações para cada meio e uso qualificadores para injetar a implementação correta.

Usando CDI, posso criar uma anotação `@Payment` e usar `@Qualifier` com `@Retention(RUNTIME)` e `@Target(...)`. Assim, ao definir um `@Payment(type = PaymentType.PAYPAL)` e `@Payment(type = PaymentType.CREDIT_CARD)`, o container consegue identificar automaticamente qual implementação deve ser usada. Isso facilita muito a extensão. Se eu quiser adicionar outro tipo de pagamento, é só criar uma nova implementação com o tipo correspondente.

Outro recurso bacana é o uso de `AnnotationLiteral` para criar filtros dinâmicos. Isso permite, por exemplo, selecionar dinamicamente a implementação com base em alguma lógica de negócio, como o país do cliente. O enum `PaymentType` pode ter um método estático que define qual tipo deve ser usado com base em um país, por exemplo.

Agora, indo para o Observer, esse pattern também é extremamente poderoso. O CDI tem um suporte riquíssimo para eventos. Em vez de acoplar diretamente quem gera e quem consome um evento, posso usar o `Event<T>` do CDI e disparar eventos com `fire()`. Os consumidores apenas anotam métodos com `@Observes` para reagir. Isso garante desacoplamento total.

No exemplo que mostrei, criei uma classe `News`, e vários consumidores como `Newspaper`, `Magazine` e `SocialMedia`, cada um ouvindo o mesmo evento. Quando o evento é disparado, todos são notificados automaticamente. É muito mais elegante do que sair adicionando manualmente os ouvintes dentro de um service.

O Builder é outro padrão interessante, que mencionei anteriormente. É ótimo para criar objetos complexos sem precisar de construtores gigantescos. Ele também ajuda a aplicar validações na criação e melhora a legibilidade do código.

Agora vamos ao Decorator. Ele é excelente para adicionar responsabilidades a objetos sem usar herança. Usei o exemplo do Worker, que tem uma implementação básica com um Programmer, e depois criei um Manager como decorador. O manager executa tarefas extras, como enviar relatórios, além da tarefa principal. Com CDI, é só anotar com `@Decorator` e delegar a chamada para o objeto original usando `@Delegate`.

O Proxy é outro padrão que aparece naturalmente no CDI e Spring. Ele é usado para controlar acesso, fazer logging, caching, e lazy loading. O CDI se destaca por permitir proxies *reflectionless*, o que torna a execução mais leve e eficiente.

Minha dica aqui é: ao invés de decorar manualmente o código com if-else ou usar herança desnecessária, use os recursos do Java com boas práticas de design. Pense em como dividir responsabilidades, aplicar coesão e desacoplamento. Quando você faz isso bem, os patterns aparecem naturalmente.

Por fim, quero reforçar que a melhor forma de aprender Design Patterns é aplicando. Você pode começar com o problema, usar boas práticas e, quando perceber, estará usando um pattern. Depois é só dar nome a ele. O importante é entender o *porquê* por trás de cada escolha.

Espero que essas aplicações tenham te ajudado a enxergar Design Patterns como algo natural, prático e extremamente útil. Vamos em frente!

Capítulo 11 - Visão Geral Sobre DDD:

Olá pessoal, bom dia, boa tarde ou boa noite! Chegamos à última etapa desse módulo, e sim, vamos falar sobre DDD, ou Domain Driven Design. E quando falo em DDD, não é sobre o código de área da Embratel, mas sim sobre desenvolvimento orientado a domínio.

Essa é uma daquelas abordagens que muita gente conhece de nome, mas pouca gente aplica corretamente. Sempre que dou uma palestra ou aula sobre DDD, pergunto: quem aqui já ouviu falar de DDD? E quase todo mundo levanta a mão. Mas quando pergunto quem já aplicou corretamente ou ficou feliz com o resultado, quase ninguém responde positivamente.

O DDD é um dos assuntos com mais buzzword do nosso meio, mas infelizmente com muita desinformação também. Por isso, antes de falarmos sobre o que é

DDD, é importante deixar claro o que ele *não* é. DDD não é um framework de tecnologia. Ele não depende de Java, Spring, JPA, nem de anotações específicas. Ele é agnóstico a linguagem de programação, a banco de dados e também a microserviços. Além disso, DDD não é sinônimo de complexidade. A ideia é justamente o oposto: trazer clareza e foco ao que realmente importa no sistema.

O objetivo do DDD é transferir conhecimento do domínio do negócio para o software. E para isso, existem duas partes fundamentais: a estratégica e a tática. O maior erro que vejo é começar diretamente pela parte tática. A parte estratégica é o coração do DDD. Sem ela, você não consegue aplicar os conceitos da maneira correta. Então vamos falar sobre essa parte com mais atenção.

A primeira etapa da parte estratégica é identificar os subdomínios: core, genérico e supporting. O core é o diferencial da sua organização, é onde está a maior complexidade e onde você deve investir mais esforço. O genérico é aquilo que é comum a muitas empresas, como sistemas de pagamento. Já o supporting são funcionalidades complementares, como sistema de notificação.

Esses subdomínios variam conforme a empresa. Por exemplo, pagamento pode ser um subdomínio genérico para um e-commerce, mas é o core para uma empresa como a Stripe. Saber identificar isso é função essencial de quem atua como arquiteto ou staff engineer.

Depois, precisamos definir uma linguagem ubíqua. Isso significa criar um vocabulário comum entre desenvolvedores e especialistas do domínio. A mesma palavra pode ter significados diferentes dependendo do contexto. Definir claramente o que cada termo significa evita ruídos na comunicação e melhora a qualidade do software.

Em seguida, delimitamos os contextos de atuação, criando os chamados *bounded contexts*. Cada contexto pode ter visões diferentes sobre as mesmas entidades. Um "Customer" no contexto de vendas pode significar algo diferente do "Customer" no contexto de suporte. Com isso em mãos, podemos aplicar os *context maps*, ou mapas de contexto, que são padrões para garantir que essa

comunicação entre contextos aconteça de forma segura e sem contaminação de domínios.

Para descobrir e organizar tudo isso, temos técnicas como o *EventStorming*, muito utilizado por times de produto, e o *Domain Storytelling*, que eu particularmente gosto muito. Histórias sempre foram uma das ferramentas mais poderosas da humanidade. Com storytelling conseguimos extrair informações que nos ajudam a desenhar modelos táticos a partir das histórias contadas pelos especialistas do domínio.

A parte tática, por sua vez, envolve entidades, value objects, serviços, repositórios, agregadores, fábricas e eventos. Aqui já estamos pensando em implementação. Mas não podemos chegar aqui sem antes termos feito o dever de casa na parte estratégica.

Um ponto importante: aplicar DDD é se preparar para a mudança. Negócios mudam o tempo todo. Por isso, projetar com DDD ajuda a criar um software mais adaptável. Gosto muito de usar o JMock para criar evidências arquiteturais no código, usando anotações e validações com Mockito. Ele me ajuda a garantir que as convenções de arquitetura e os padrões definidos estão sendo seguidos.

E por fim, DDD é compatível com diversas arquiteturas: camadas, hexagonal, onion, microkernel, event-driven e outras. Você pode usar DDD com Clean Architecture sem problemas. O importante é entender que DDD não é sobre ferramentas ou frameworks, mas sobre entender o negócio, criar modelos ricos e evolutivos e manter seu software aderente à realidade.

Então é isso, pessoal. Nessa aula passamos por definições, erros comuns, parte estratégica, parte tática e vimos boas práticas que ajudam a colocar tudo isso em prática. Vamos seguir juntos para a próxima etapa!

Capítulo 12 - Conhecendo o JMolecules:

Fala pessoal! Agora que a gente já explorou bastante sobre DDD, quero apresentar para vocês uma ferramenta que tem me ajudado muito a validar os conceitos arquiteturais no código: o JMolecules. Essa biblioteca traz um conjunto de anotações que permite colocar evidências arquiteturais diretamente no seu modelo de domínio. E mais do que isso, ela funciona perfeitamente com o ArcUnit, o que significa que você consegue escrever testes que validam a estrutura arquitetural do seu sistema. Isso é incrível.

A primeira coisa que você vai perceber ao usar o JMolecules é que ele te obriga a explicitar suas intenções arquiteturais. Por exemplo, ao criar uma entidade, você pode usar a anotação `@Entity` do JMolecules e, com isso, informar que aquele objeto possui uma identidade própria dentro do seu domínio. É muito comum esquecermos de marcar o atributo que representa essa identidade, mas

o JMolecules vai te lembrar disso. Ele reclama se não tiver nenhum campo com a anotação `@Identity`, por exemplo.

Vamos supor que estou modelando um cartão de crédito. Eu crio uma classe `CreditCard`, marco com `@Entity` e, naturalmente, preciso ter o `id`, o `maskedNumber` e outras informações relevantes. E como é comum nesse tipo de sistema, também vou precisar de objetos do tipo `enum`, como por exemplo o tipo do cartão (`Visa`, `MasterCard`). Esses `enums`, quando usados para representar conceitos do domínio sem identidade própria, são considerados `Value Objects`. No JMolecules, você também pode marcar esses elementos com `@ValueObject`, deixando isso claro no seu modelo.

Outro ponto interessante é o conceito de agregados. Quando tenho uma `Order` que agrega uma lista de `Product`, essa `Order` passa a ser o `aggregate root`. E o JMolecules ajuda você a deixar isso evidente através de anotações como `@AggregateRoot`. Isso é ótimo para reforçar os limites do domínio e organizar melhor as dependências.

A diferença entre `DAO` e `Repository` também fica mais clara quando você usa JMolecules. Um `DAO` está ligado à abstração do banco de dados, com métodos como `insert`, `update`, `delete`. Já o `Repository` representa uma coleção do domínio e suas operações fazem parte da linguagem ubíqua. Por exemplo, posso ter um `CreditCardRepository` com métodos como `save` ou `order`, dependendo do contexto de negócio. Isso ajuda a manter o código mais expressivo e aderente à realidade do domínio.

No meu serviço de cartão, posso usar a anotação `@Service` do JMolecules para marcar que aquela classe é responsável por coordenar operações do domínio. Além disso, o JMolecules é totalmente compatível com o Spring, o que significa que você pode combiná-lo com outras anotações do ecossistema sem problema algum.

Outra funcionalidade muito bacana é a separação em camadas. Com o JMolecules, você consegue anotar suas classes com `@ApplicationLayer`, `@DomainLayer`, `@InfrastructureLayer` e `@InterfaceLayer`. Isso facilita muito a

visualização da arquitetura e ajuda a evitar misturas indevidas entre responsabilidades.

E, claro, com o suporte ao ArcUnit, você consegue escrever testes que garantem que essas anotações estão sendo seguidas corretamente. Isso torna o design do sistema algo verificável, mensurável e confiável.

Eu particularmente prefiro usar as anotações do que implementar interfaces, porque acho que deixa o código mais limpo e objetivo. E o mais legal é que a documentação do JMolecules é rica, cheia de exemplos, explicações e links úteis para você se aprofundar.

Espero que vocês tenham curtido essa apresentação do JMolecules. Ele é uma ferramenta poderosa para aplicar os conceitos de DDD e garantir a integridade arquitetural do seu projeto. Se tiverem dúvidas, me procurem. Até a próxima!

